

# **AI for Economic Research: Dynamic Models, Language, and Agents**

## **Lecture 8.1: Where We Are, Naive Prompting, and Personal Wiki KBs**

Zhigang Feng

Spring 2026

## Topic 8.1 Agenda

1. **Where We Are** — recap of LLM limits
2. **Naive Prompt Engineering Alone** — limits without retrieval
3. **Personal Wiki Knowledge Bases** — the pre-formalism approach

*Arc: T1 Prompting wall → T2 Chunking & embeddings → T3 Vector & graph retrieval → T4a Augmented generation → T4b Demo, failures, agentic bridge*

## **Where We Are**

---

## Recap: What an LLM Is (Lecture 7)

- A **Large Language Model** is a neural network that maps a sequence of tokens to a probability distribution over the next token.
- **Pipeline:** Tokenization → Token embeddings → Transformer layers (attention + feed-forward) → Output distribution.
- **Knowledge is parametric:** Everything the model “knows” is encoded in its weights, frozen at the end of pretraining.
- **Implication:** The model has no memory beyond its weights and the current context window. Once training is over, its knowledge of the world is fixed in time.
- **Today’s question:** How do we extend that knowledge *at inference time*, without retraining?

# What an LLM Knows vs. What It Can Know

- Every LLM has a **knowledge cutoff**: the date when its training corpus stopped.
  - GPT-4 Turbo: April 2023
  - GPT-4o: October 2023
  - Claude 3.5 Sonnet: April 2024
  - Claude 4 Opus: early 2025
- **Concrete problem:** Ask the model: *“What did the FOMC decide at its December 2025 meeting?”*
  - A pre-2025 model has *never seen* the December 2025 statement.
  - It will either refuse, or worse, *hallucinate* a plausible but fictional answer.
- **The economist’s view:** Macro and finance research depends on the latest data. A model that can’t read the December press release is not a research tool.
- **Even within the cutoff:** Private datasets (your own working papers, internal Fed memos, FOIA-released records) were never in the training corpus at all.

## Three Pain Points for the Economist

- **1. Stale knowledge.** Anything published after the cutoff is invisible. So is anything proprietary.
- **2. Hallucinated citations.** A vanilla LLM asked to “cite five NBER papers on the slope of the Phillips curve” will happily produce five plausible titles, plausible authors, and entirely fake DOIs. This is the most common failure mode in academic use.
- **3. No source attribution.** Even when the answer is correct, the model cannot tell you *where* the claim came from. Academic research requires footnotes.
- **Bottom line:** For research-grade work we need a system that can (i) read documents the model never saw, (ii) refuse to invent facts, and (iii) tell us which document supplied each claim.

## Two Ways to Add Knowledge

- **Option A — Fine-Tuning:** Re-train (some of) the model's weights on your new corpus.
  - Pros: knowledge becomes “baked in,” fast at inference.
  - Cons: expensive, fragile, hard to update, no source attribution, doesn't fix hallucination.
- **Option B — Retrieval (RAG):** Leave the model untouched. At query time, fetch the relevant text from an external store and *paste it into the prompt*.
  - Pros: cheap, updatable in real time, gives source attribution, dramatically reduces hallucination.
  - Cons: more moving parts; quality depends on the retriever.
- **Promise of this lecture:** By the end you will understand both options, know when to use each, and have seen the full RAG pipeline applied to FOMC and NBER corpora.
- **Detailed comparison:** Section VIII.

## **Naive Prompt Engineering Alone**

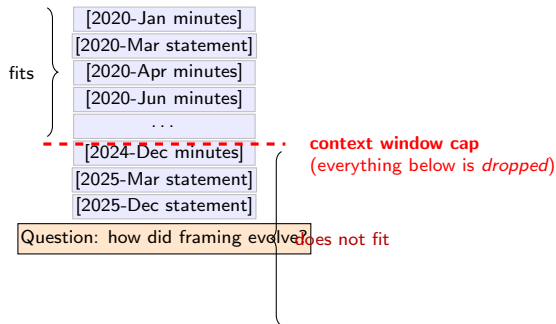
---



# Naive Prompt Engineering: A Tempting First Attempt

**Setup** (Lec. 7 T3a covered prompting itself — zero-shot, few-shot, in-context learning). The naive RAG-less workflow goes one step further: paste the *whole corpus* into the prompt and let the model read it directly.

**Research question:** “How did the FOMC’s framing of inflation expectations evolve from 2020 to 2025?”



Four reasons it still fails:

1. context window (red dashed line)
2. cost — pay every token, every query
3. “lost in the middle” (Lec. 7 T3b)
4. no persistence across sessions

*In principle the model has all the data it needs. In practice four hard walls block this approach — we* 17

## Limit #1: The Context Window

- Every LLM has a hard cap on how many tokens it can read at once — the **context window**.
  - Claude Opus 4.8: 1,000,000 tokens (about 750,000 English words; 1M context is standard since Opus 4.6)
  - GPT-5.4 / GPT-5.5:  $\sim 1,000,000$  tokens (current OpenAI frontier)
  - GPT-4 Turbo: 128,000 tokens (legacy; not the current frontier)
  - Gemini 2.5 Pro: 1,000,000 tokens

*Model facts current as of June 2026.*

- **How big is the FOMC corpus?**
  - 8 meetings per year  $\times$  statement ( $\sim 1,000$  tokens) + minutes ( $\sim 15,000$  tokens) + transcript ( $\sim 100,000$  tokens, released with 5-year lag)
  - Conservative estimate *excluding* transcripts:  $\sim 130,000$  tokens *per year*
  - Five years (2020–2024):  $\sim 650,000$  tokens
- **Verdict:** Five years of FOMC text *does not fit* in any model's context window. And we haven't even added press conferences, speeches, or Beige Books.
- **This is a hard wall, not a tuning parameter.**

## Limit #2: Cost (Callback to Lec. 7 T3a)

- Even when content fits, you pay for *every input token, every query*. Lec. 7 T3a “Tokens, Cost, and Reproducibility” lists the per-token rates for the frontier models (Opus, GPT-4o, Gemini, local Llama proxy).
- **RAG-specific cost arithmetic.** Stuff 600K tokens of FOMC text into each query (input-token prices):
  - One query on Claude Opus:  $0.6 \times \$5 = \$3.00$
  - One query on GPT-4o:  $0.6 \times \$2.50 = \$1.50$
  - One thousand exploratory queries:  $\$3,000 / \$1,500$

*Claude Opus input price is \$5/M (output \$25/M) since Opus 4.5; model facts current as of June 2026.*

- **You pay this every time**, even on a repeat question. There's no caching of expensive context across sessions.
- **For research:** prohibitive at the scale of a literature review or an empirical project. (RAG instead pays  $O(N)$  embedding calls *once* — T4a develops the visual.)

### Limit #3: Latency and “Lost in the Middle” (Callback to Lec. 7 T3b)

- **Latency.** Inference time grows roughly linearly (or worse) in input token count. A 600K-token query can take 30+ seconds before the first response token.
- **Quality degrades, too.** Lec. 7 T3b “Lost in the Middle: Positional Saliency in Long Contexts” established the mechanism — under Liu et al. (2024, *TACL*), recall on relevant context drops 20–30 pp. when the evidence sits in the middle of the prompt; the position–accuracy curve is U-shaped.
- **Implication here:** even if the December 2025 statement *did* fit, concatenated between four years of prior minutes it would land in the middle and be systematically under-weighted.
- **Punchline for RAG motivation:** cramming everything into the prompt is not just expensive — it actively *hurts* answer quality. RAG must therefore both *retrieve* and *rank* chunks, not merely retrieve.

## Limit #4: No Scalability or Persistence

- Naive prompt engineering treats every query as a **fresh start**.
  - No reuse of work across queries.
  - No incremental updates when the Fed releases new minutes next month.
  - No way to grow the corpus past the context window — ever.
- **Compare to a real research workflow:**
  - You collect documents *once*.
  - You index them *once*.
  - You ask many questions over time, paying only for what you actually retrieve.
  - When new documents arrive, you index them incrementally.
- Naive prompt engineering supports *none* of this. Every question is a brand-new copy-paste.
- **The pattern we need:** Separate the cost of *ingesting documents* from the cost of *asking questions*.

# The Wall

| Naive prompt engineering can do                  | Research workflows actually need       |
|--------------------------------------------------|----------------------------------------|
| QA on a single short document                    | QA over millions of tokens of corpus   |
| Tens of queries before cost bites                | Thousands of queries over weeks        |
| Whatever fits in the context window (~1M tokens) | Decades of FOMC + NBER + 10-K filings  |
| Today's prompt only                              | Incremental updates as new docs arrive |
| "Trust me" answers                               | Cited, source-attributed answers       |

**Conclusion:** Prompt engineering hits a hard wall on freshness, scale, latency, cost, and attribution. We need a fundamentally different architecture — one that **separates indexing from querying**.

*This is what RAG does.*

## **Before the Formalism: Personal Wiki Knowledge Bases**

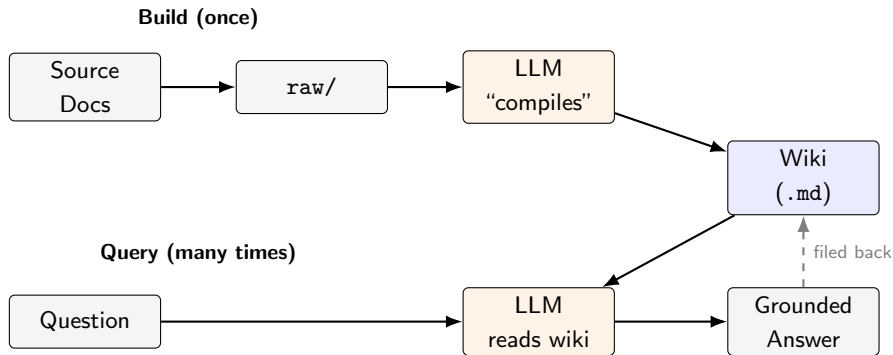
---

# Before the Formalism: A Personal Wiki

- Before we build the full RAG pipeline, consider a **simpler version** of the same idea that you can set up today.
- **Andrej Karpathy** (June 2025):  
*"I'm finding very useful recently: using LLMs to build personal knowledge bases for various topics of research interest. [...] A large fraction of my recent token throughput is going less into manipulating code, and more into **manipulating knowledge** (stored as markdown and images)."*
- **The workflow is disarmingly simple:**
  1. Collect source documents (papers, articles, data) into a `raw/` directory.
  2. Ask an LLM to *compile* them into a **wiki** — a directory of `.md` files with summaries, concept articles, and cross-links.
  3. Query the wiki by asking the LLM questions; it reads the relevant articles and answers from them.
- **Why this matters for us:** This is the *same core insight* behind RAG — separating data ingestion from querying — but without any of the formal machinery.



# The Wiki Pipeline



- **Data ingest:** Download papers, clip web articles, save datasets → raw/.
- **Compile:** LLM reads raw data, produces wiki articles — summaries, concept pages, back-links. *You rarely edit the wiki manually; the LLM maintains it.*
- **Query:** Ask the LLM a question. It reads the wiki's index and relevant articles, then answers with grounded citations.
- **Feedback loop (dashed):** Answers and explorations get filed back into the wiki, so your

## Wiki vs. RAG: Same Intuition, Different Scale

|                    | Personal Wiki                                  | Full RAG Pipeline                       |
|--------------------|------------------------------------------------|-----------------------------------------|
| <b>Ingestion</b>   | LLM reads raw docs, writes .md summaries       | Chunk → embed → store in vector DB      |
| <b>“Retrieval”</b> | LLM reads index files, picks relevant articles | Embedding similarity search (top- $K$ ) |
| <b>Generation</b>  | LLM answers from wiki articles in context      | LLM answers from retrieved chunks       |
| <b>Scale limit</b> | ~400K words (must fit in context)              | Millions of tokens (no upper limit)     |
| <b>Update</b>      | LLM rewrites/extends articles                  | Add new chunks incrementally            |
| <b>Attribution</b> | Article titles and links                       | Chunk IDs and source metadata           |

- Karpathy: *“I thought I had to reach for fancy RAG, but the LLM has been pretty good about auto-maintaining index files and brief summaries [...] at this ~small scale.”*

# Hands-On: A Mini Wiki for This Course

```
course_wiki/  
  index.md                -- master table of contents  
  concepts/  
    embeddings.md         -- text-to-vector mappings (Lec 7-8)  
    rag.md                -- retrieval augmented generation  
    transformers.md       -- the architecture behind LLMs  
  lectures/  
    lec07_llm.md          -- key ideas from Lecture 7  
    lec08_rag.md          -- key ideas from Lecture 8  
  papers/  
    vaswani_2017.md       -- Attention Is All You Need  
    lewis_2020_rag.md      -- RAG original paper (NeurIPS)  
  connections/  
    cross_references.md    -- links across lectures
```

- A student asks: *“How do the embeddings from Lecture 7 connect to vector stores in Lecture 8?”* — the LLM reads the relevant wiki pages and gives a **grounded, cross-referenced** answer.
- As the course progresses, **every new lecture enriches the wiki**. Your questions “add up” instead of disappearing after each chat session.

# From Wiki to RAG: Why We Need the Formal Machinery

- The personal wiki works beautifully **at small scale**:  $\sim 100$  articles,  $\sim 400\text{K}$  words. A modern LLM can read the whole index plus relevant articles in one context window.
- **But our economics corpora are much larger**:
  - 30 years of FOMC minutes + transcripts: millions of tokens
  - NBER working paper archive: tens of thousands of papers
  - Congressional Record: billions of words
- **The progression is natural**:
  1. **Naive prompting** (paste docs into the prompt)  $\rightarrow$  hits context window at  $\sim 200\text{K}$  tokens
  2. **Personal wiki** (LLM-curated summaries)  $\rightarrow$  hits context window at  $\sim 400\text{K}$  words
  3. **RAG** (vector search over embeddings)  $\rightarrow$  scales to millions of documents
- Each step is a natural response to the **scaling problem**. RAG is the wiki idea *formalized and automated*: instead of an LLM maintaining an index by hand, we use embeddings and similarity search to do retrieval at arbitrary scale.
- Karpathy: *"I think there is room here for an incredible new product instead of a hacky collection of scripts."*
- **Next (Topic 8.2)**: the principled solution — chunking, embeddings, and a vector store that <sup>17 / 17</sup>